

UNIVERSIDADE D
COIMBRA

QCML2- Image classification with Hybrid Quantum Convolutional Neural Networks

Results

Members:
Miguel Castela 2022212972
Supervised by Professor Doctor Jorge Lobo

March, 2025

Table of Contents

Introduction 2
First evaluations with the Qiskit tutorials 2
    First tutorial 2
    Second tutorial 7
Diferent types of QNN 12
Statistics and running on real hardware 12
    Trying to manually process the images using NEQR and FRQI 12
What works best and creating a custom HCQNN 13
Conclusion 13

Introduction

Artificial neural networks are currently one of the most popular models used in machine learning. A subcategory of these called convolutional neural networks (CNN) is par- ticularly popular. These are networks mainly used for the task of image recognition. Using pooling and convolution layers they extract structured information about images. THis project will be related to CNN topic but also to Quantum computing. Quantum machine learning leverages the principles of quantum mechanics to process information, allowing for the creation of algorithms that can potentially outperform classical counterparts. By harnessing quantum bits or qubits, these algorithms can explore multiple possibilities simultaneously, offering a unique approach to solving complex problems. While still in the early stages of development, quantum machine learning holds promise for revolutionizing certain computational tasks, particularly those involving large-scale optimization and data analysis.

First evaluations with the Qiskit tutorials

First tutorial

The first step of this project was using the Qiskit framwoerk, combined with pythorch to construct a HQNN. After studying the first three qiski tutorials ("Quantum Neural Networks", "Neural Network Classifier" & "Training a Quantum Model on a Real Dataset") i began to implement the first version of the QNN. This was done by implementing the "Torch Connector and Hybrind QNNs" tutorialm:

- The first step is to initialize the Estimator. The Estimator is a class in Qiskit that allows you to estimate the expectation values of quantum circuits. It is used to evaluate the performance of quantum models by measuring their outputs.

```
estimator = Estimator()
```

- I then, using torchvision, load the MNIST training dataset(The MNIST database is a large database of handwritten digits) and apply a transformation to convert images to tensors.
  A tensor is a generalization of vectors and matrices to any number of dimensions. They are the fundamental data structure in PyTorch, similar to NumPy arrays but with additional capabilities for GPU acceleration.

```
X_train = datasets.MNIST(
    root="./data", train=True, download=True,
    transform=transforms.Compose([transforms.ToTensor()])
)
```

- I filter the dataset to keep only digits labeled 0 and 1 (first `n_samples` of each)
  I use the `np.append` function to concatenate the indices of the two classes (0 and 1) and then filter the dataset using these indices. The `np.where` function returns

the indices of elements in an array that satisfy a given condition.
The `X_train.data` and `X_train.targets` attributes are then updated to include only the selected samples.
The `X_train.data` attribute contains the image data, while `X_train.targets` contains the corresponding labels.

```
idx = np.append(
    np.where(X_train.targets == 0)[0][:n_samples]
    , np.where(X_train.targets == 1)[0][:n_samples]
)
X_train.data = X_train.data[idx]
X_train.targets = X_train.targets[idx]
```

- I define the PyTorch DataLoader for the filtered training data, this is used to load the data in batches during training. The `shuffle=True` argument ensures that the data is shuffled before each epoch, which helps improve the model's generalization.

```
train_loader = DataLoader(X_train, batch_size=batch_size, shuffle=True)
```

- I load the MNIST dataset again to create the test set, this time with `train=False` to indicate that we want the test set. The same transformation is applied as before.
I again filter the test dataset to keep only digits labeled 0 and 1 (first `n_samples` of each)
and define the DataLoader for the filtered test data.

```
n_samples = 50
X_test = datasets.MNIST(
    root="./data", train=False, download=True,
    transform=transforms.Compose([transforms.ToTensor()])
)

...
```

- Then I create the quantum neural network. At this point, a key decision must be made: **how to encode the classical input features into a quantum state** — a process known as *feature mapping* or *data encoding*.

  Qiskit provides several options for this, the most common being:

  - **ZFeatureMap**
    A simple quantum feature map where each input value is encoded as a rotation on a single qubit using Rz gates. This approach does **not include entanglement** and is typically used for simpler datasets or models with lower computational complexity.

– **ZZFeatureMap**
A more expressive feature map that includes both single-qubit rotations and **entangling gates (ZZ interactions)** between qubits. This allows the model to capture complex relationships between features, making it more suitable for richer or more structured datasets.

These feature maps are not just about storing the features — they are essential for **embedding classical data (such as pixel values from images)** into the quantum circuit so that the quantum neural network can process it.

Other alternatives to these include:

– **PauliFeatureMap** – a generalized feature map that supports different combinations of Pauli operators.

– **Custom feature maps** – where users design their own encoding strategies tailored to specific datasets or problems. This was explored in the NEQR and FRQI section

The choice of feature map directly influences the expressiveness and performance of the quantum model. In this case, i used the `ZZFeatureMap`.

The next choice is the **ansatz** (the quantum circuit structure). The ansatz defines the architecture of the quantum neural network, including the number of qubits, layers, and gates used. Qiskit provides several predefined ansatz classes, such as:

– **EfficientSU2**
A popular ansatz that uses a combination of single-qubit rotations and entangling gates. It is designed to be efficient in terms of circuit depth and is suitable for a wide range of problems.

– **RealAmplitudes**
This ansatz uses real-valued parameters and is often used for variational algorithms. It can be more expressive than EfficientSU2 but may require more qubits and gates.

– **TwoLocal**
A flexible ansatz that allows users to specify the types of gates and the connectivity between qubits. It can be tailored to specific problems but may require more tuning.

The choice of ansatz is crucial as it determines the expressiveness and complexity of the quantum model. In this case i used the RealAmplitudes.

```
def create_qnn():
    feature_map = ZZFeatureMap(2)
    ansatz = RealAmplitudes(2, reps=1)
    qc = QuantumCircuit(2)
    qc.compose(feature_map, inplace=True)
    qc.compose(ansatz, inplace=True)

    # REMEMBER TO SET input_gradients=True FOR ENABLING HYBRID GRADIENT BACKPR
    qnn = EstimatorQNN(
```

```
        circuit=qc,
        input_params=feature_map.parameters,
        weight_params=ansatz.parameters,
        input_gradients=True,
        estimator=estimator,
    )
    return qnn

qnn4 = create_qnn()
```

- The next step is to define the neural network that includes a quantum layer.

  This class Net is a custom neural network built using PyTorch. It combines classical layers (such as convolutional and linear layers) with a quantum neural network (QNN) layer created earlier with Qiskit. The QNN is integrated using the TorchConnector, which allows it to be used just like any other PyTorch layer and supports hybrid quantum-classical training via backpropagation.

  There are two convolutional layers, typically used for image data.

  $Conv2d(1, 2, kernel_size = 5)$ means the first layer takes 1 input channel ($grayscaleimage$) and outputs 2 feature maps using 5x5 filters.

  The second layer takes 2 channels and produces 16 feature maps.

  Dropout2d is a regularization layer that randomly zeros out entire channels during training to prevent overfitting.

  fc1 and fc2 are fully connected (linear) layers. The first one takes the flattened output of the convolutional layers (256 features) and reduces it to 64 features. The second one takes the 64 features and outputs 2 features, which are then passed to the QNN.

  The Qiskit quantum neural network is then wrapped using the TorchConnector, allowing it to be used as a layer in the PyTorch model. The QNN takes the 2-dimensional input from fc2 and produces a 1-dimensional output.

  The parameters (weights) of the quantum circuit are initialized randomly in the interval $[-1, 1]$.

  Finally, fc3 is another fully connected layer that takes the 1-dimensional output from the QNN and produces a final output of 2 features (which can be interpreted as probabilities for binary classification).

```
class Net(Module):
def __init__(self, qnn):
    super().__init__()
```

```
self.conv1 = Conv2d(1, 2, kernel_size=5)
self.conv2 = Conv2d(2, 16, kernel_size=5)
self.dropout = Dropout2d()
self.fc1 = Linear(256, 64)
self.fc2 = Linear(64, 2)  # 2-dimensional input to QNN
self.qnn = TorchConnector(qnn)  # Apply torch connector, weights chose
# uniformly at random from interval [-1,1].
self.fc3 = Linear(1, 1)  # 1-dimensional output from QNN
```

- The next step is to do the Forward pass.

  It applies the first convolutional layer with a ReLU activation, followed by 2x2 max pooling (downsampling). The same steps are repeated for the second convolutional layer.

  After feature extraction with convolutional layers and dropout for regularization, the data is flattened and passed through two fully connected layers (fc1 and fc2). The output, a 2D vector, is then processed by the quantum neural network (QNN) layer. Its result goes through another linear layer (fc3), and the final output is a concatenation of the prediction and its complement, representing probabilities for binary classification (digit 0 vs 1).

```
def forward(self, x):
x = F.relu(self.conv1(x))
x = F.max_pool2d(x, 2)
x = F.relu(self.conv2(x))
x = F.max_pool2d(x, 2)
x = self.dropout(x)
x = x.view(x.shape[0], -1)
x = F.relu(self.fc1(x))
x = self.fc2(x)
x = self.qnn(x)  # apply QNN
x = self.fc3(x)
return cat((x, 1 - x), -1)
```

- Finally i need to inicialize the mode, optimizer and loss function.
  The optimizer is responsible for updating the model's parameters during training. In this case, the Adam optimizer is used with a learning rate of 0.01.
  The loss function is used to measure the difference between the predicted output and the true labels. In this case, the Negative Log Likelihood Loss is used.

  Adam is a first-order gradient-based optimizer that combines momentum, which uses past gradients, and adaptive learning rates, which scale the step size based on how frequently a parameter is updated. It is commonly used for most deep learning tasks due to its speed and robustness. One of its key advantages is the adaptive learning rate per parameter, which often leads to good convergence. However, one downside of Adam is that it can sometimes generalize worse than Stochastic Gradient Descent (SGD).

The model is trained for 10 epochs. In each epoch, it iterates over the training batches:

  - Gradients are reset with `optimizer.zero_grad()`.
  - A forward pass is performed to get the model's predictions.
  - The loss between predictions and true labels is computed.
  - A backward pass computes gradients.
  - The model updates its weights using the optimizer.
  - Loss values for the epoch are stored and averaged.

- A new quantum neural network is created and the trained weights are loaded:

  The model is then evaluated on the test dataset. First, it is set to evaluation mode using `model.eval()`. Gradient computation is disabled using `no_grad()` to speed up inference and save memory.

  For each batch in the test set:

  - The model outputs a prediction for the input data.
  - If the output has an unexpected shape, it is reshaped appropriately.
  - The predicted label is obtained using `argmax`.
  - The number of correct predictions is counted by comparing with the true labels.
  - The loss is computed using the same loss function as in training.

  At the end, the average loss and accuracy are printed

Second tutorial

The second tutorial added the concepts of pooling layers, creating the pooling and convolution circuits and a custom dataset.

- Quantum Circuit Definitions

  A two-qubit unitary circuit is defined for the convolutional layer. The circuit is parameterized and drawn.

```
def conv_circuit(params):
    target = QuantumCircuit(2)
    target.rz(-np.pi / 2, 1)
    target.cx(1, 0)
    target.rz(params[0], 0)
    target.ry(params[1], 1)
    target.cx(0, 1)
    target.ry(params[2], 1)
    target.cx(1, 0)
```

```
    target.rz(np.pi / 2, 0)
    return target
```

The convolutional circuit is then drawn and saved as an image.

- Convolutional Layer A convolutional layer is constructed by composing multiple convolutional circuits. The result is a quantum circuit that can be applied to the qubits.

```
def conv_layer(num_qubits, param_prefix):
    qc = QuantumCircuit(num_qubits, name="Convolutional Layer")
    qubits = list(range(num_qubits))
    param_index = 0
    params = ParameterVector(param_prefix, length=num_qubits * 3)
    for q1, q2 in zip(qubits[0::2], qubits[1::2]):
        qc = qc.compose(conv_circuit(params[param_index : (param_index + 3)]), [q1
        qc.barrier()
        param_index += 3
    for q1, q2 in zip(qubits[1::2], qubits[2::2] + [0]):
        qc = qc.compose(conv_circuit(params[param_index : (param_index + 3)]), [q1
        qc.barrier()
        param_index += 3

    qc_inst = qc.to_instruction()

    qc = QuantumCircuit(num_qubits)
    qc.append(qc_inst, qubits)
    return qc
```

- Pooling Circuit A similar approach is used to define a pooling layer, which uses a different set of operations.

  A pooling layer's objective is to reduce the dimensionality of the data while retaining important features. This is done by applying a pooling circuit that combines qubits in a specific way.

```
def pool_circuit(params):
    target = QuantumCircuit(2)
    target.rz(-np.pi / 2, 1)
    target.cx(1, 0)
    target.rz(params[0], 0)
    target.ry(params[1], 1)
    target.cx(0, 1)
    target.ry(params[2], 1)
    return target
```

- Pooling Layer The pooling layer is created by composing several pooling circuits.

```
def pool_layer(sources, sinks, param_prefix):
    num_qubits = len(sources) + len(sinks)
    qc = QuantumCircuit(num_qubits, name="Pooling Layer")
    param_index = 0
    params = ParameterVector(param_prefix, length=num_qubits // 2 * 3)
    for source, sink in zip(sources, sinks):
        qc = qc.compose(pool_circuit(params[param_index : (param_index + 3)]), [sou
        qc.barrier()
        param_index += 3

    qc_inst = qc.to_instruction()

    qc = QuantumCircuit(num_qubits)
    qc.append(qc_inst, range(num_qubits))
    return qc
```

- Dataset Generation A synthetic dataset of images is created to train the QCNN. Each image is either a horizontal or vertical line.
- Model Training and Evaluation

  The dataset is split into training and testing sets. The classifier is then trained using a quantum neural network with the defined convolutional and pooling layers.

```
train_images, test_images, train_labels, test_labels = train_test_split(
    images, labels, test_size=0.3, random_state=246
)

classifier.fit(x, y)
```

- Define the ansatz and feature map:

  We start by initializing a feature map (ZFeatureMap) and a quantum circuit (ansatz) that will serve as our ansatz. This circuit is responsible for transforming input data into a quantum state.

- Define the quantum circuit:
  - **First Convolutional Layer:**
    - A convolutional layer is composed on the ansatz. It operates on all 8 qubits and uses a set of parameters (denoted "c1") to define the unitary operations.
  - **First Pooling Layer:**
    - A pooling layer is added after the first convolutional layer. This pooling layer combines qubits from the first half ([0, 1, 2, 3]) with those from the second half ([4, 5, 6, 7]), creating a reduced representation of the data.
  - **Second Convolutional Layer:**

    - Another convolutional layer is added, this time operating on the 4 qubits from the second half of the previous layer ([4, 5, 6, 7]), and using the "c2" parameter set.
  - **Second Pooling Layer:**
    - After the second convolutional layer, another pooling layer is applied, reducing the qubit set further to [0, 1] and [2, 3].
  - **Third Convolutional Layer:**
    - A third convolutional layer operates on the 2 remaining qubits from the second pooling layer ([6, 7]), with the "c3" parameter set.
  - **Third Pooling Layer:**
    - A final pooling layer is added, operating on the single remaining pair of qubits [0] and [1].

  After creating the convolutional and pooling layers, they are combined with the feature map in the circuit. The feature map is applied first, followed by the ansatz to complete the transformation.

- Define the observable

  The observable is defined as a SparsePauliOp with the operator ("Z" + "I" * 7), which will measure the Z component of the state on the first qubit while leaving the others unchanged. This observable is used to extract information from the quantum state after the ansatz has been applied.

```
observable = SparsePauliOp.from_list([("Z" + "I" * 7, 1)])
```

- Decompose the circuit for the QNN:

  The circuit is decomposed to avoid data duplication when passed to the quantum neural network (QNN). This ensures the circuit is executed efficiently in the EstimatorQNN model.

```
qnn = EstimatorQNN(
circuit=circuit.decompose(),
observables=observable,
input_params=feature_map.parameters,
weight_params=ansatz.parameters,
estimator=estimator,
)
```

- Initialize and train the classifier:

  The NeuralNetworkClassifier is initialized with the QNN and the $COBYLA$ optimizer (with a maximum of 200 iterations). A callback function (callback_graph) is passed to track the objective function during training.

  The classifier is then trained using the training data (x, y).

```
# Initialize the classifier
initial_point = np.random.random(qnn.num_weights)
classifier = NeuralNetworkClassifier(
    qnn,
    optimizer=COBYLA(maxiter=200),  # Set max iterations here
    callback=callback_graph,
    initial_point=initial_point,
)

# Convert training data to numpy arrays
x = np.asarray(train_images)
y = np.asarray(train_labels)

# List to store objective function values
objective_func_vals = []
plt.rcParams["figure.figsize"] = (12, 6)

# Train the classifier
classifier.fit(x, y)
```

During classifier.fit(), the following occurs:

The NeuralNetworkClassifier performs a forward pass by computing the output for each input image x in `train_images`. The data is passed through the quantum circuit defined by `feature_map` and `ansatz`, and the output is compared to the true labels y. The quantum neural network (qnn) is a parameterized model, and this forward pass generates predictions by applying the quantum operations defined in the ansatz to the data encoded in the quantum feature map.

Backward Pass:

The backward pass involves calculating the gradients of the loss function with respect to the parameters (weights) of the network and updating those parameters to minimize the loss. This is where the optimization process happens. In this case, the optimizer is COBYLA, which is a gradient-free optimization algorithm. While COBYLA does not explicitly require gradients (as it doesn't rely on gradient-based updates), it still operates by iterating through the optimization process to find the best set of weights for the QNN by minimizing the objective function.

Differenced betweenn Adam and COBYLA:

- **Adam Optimizer**:
  - **Momentum**: Tracks the exponentially decaying average of past gradients to help avoid oscillations and speed up convergence.
  - **Adaptive Learning Rates**: Adjusts the learning rates for each parameter based on past gradients, improving convergence rates for different parameters in neural networks.
  - **Bias Correction**: Corrects the initial bias in the moment estimates, especially for small t (iteration steps).

- **Key Hyperparameters of Adam**:
  - `alpha`: Learning rate.
  - `beta1`: Exponential decay rate for the first moment estimate.
  - `beta2`: Exponential decay rate for the second moment estimate.
  - `epsilon`: Small constant to prevent division by zero.
- **COBYLA Optimizer**:
  - **Derivative-Free**: COBYLA is a derivative-free optimization method suitable for problems without gradients or where gradient calculation is expensive.
  - **Key Difference from Adam**: Unlike Adam, COBYLA does not rely on gradients, making it suitable for non-differentiable problems.
- **Alternative to `optimizer.zero_grad(set_to_none=True)`**:
  - COBYLA does not require clearing gradients since it does not use gradients.
  - For gradient-based optimizers (like Adam), you typically use `optimizer.zero_grad()`, but COBYLA does not need such a step.
  - When using COBYLA in frameworks like PyTorch, avoid gradient-based methods entirely and focus on function evaluations.

- Model Evaluation:

```
y_predict = classifier.predict(test_images)
```

Diferent types of QNN

The first tutorial focuses on a direct application of a quantum neural network (QNN) in a hybrid classical-quantum neural network, using a known dataset like MNIST. The model begins with classical processing (convolutions, pooling, fully connected layers), passes through quantum circuits for feature extraction, and concludes with a classical layer for classification.

In contrast, the second tutorial adopts a more experimental approach, where data is processed classically and then converted into a quantum format. The QCNN uses convolutional and pooling layers to extract features, and the NeuralNetworkClassifier trains the model. Precision is evaluated, and results are displayed in a classical manner.

Statistics and running on real hardware

Trying to manually process the images using NEQR and FRQI

Flexible Representation of Quantum Images (FRQI): This method encodes both the color (grayscale intensity) and position of each pixel into a quantum state. Specifically, the intensity information is represented by rotating a qubit to a specific angle, while the pixel

positions are mapped using basis states.
Novel Enhanced Quantum Representation (NEQR): Unlike FRQI, NEQR uses multiple qubits to represent the grayscale value of each pixel directly, allowing for a more precise encoding of image information.
I tried to implement the NEQR and FRQI methods in the first tutorial, instead of using ZZFeatureMap. These were the functions used:

What works best and creating a custom HCQNN

Based on the results obtained from the first two tutorials, i developed a custom hybrid quantum convolutional neural network (HCQNN) that combines the best features of both approaches.

Conclusion